BRICK BREAKER GAME DEVELOPMENT USING PYTHON

AIM

To design and develop an interactive 2D Brick Breaker (Breakout) game using Python Tkinter that allows the player to control a paddle using the mouse or keyboard arrow keys, bounce a ball to destroy rows of coloured bricks, earn points, and manage a limited number of lives. The game includes a graphical interface, score and lives tracking, win and game-over detection, and restart and exit options. A browser-playable HTML version of the same game is also provided.

PROCEDURE

1. Start the program and import the required Python libraries: tkinter and random.
2. Define the game settings (window size, paddle size and speed, ball radius and speed, number of brick rows and columns, row colours) as constants.
3. Create the main game window using Tkinter and set the title as "BRICK BREAKER".
4. Design the interface with Label widgets for the title, SCORE and LIVES, and a Canvas widget as the game board with a dark background and a blue border.
5. Create the bricks as a list of (x, y, row) tuples arranged in 5 rows and 10 columns, each row having its own colour.
6. Create the paddle as a rectangle at the bottom of the board and the ball as a circle resting on the paddle.
7. Implement controls: mouse movement or the Left/Right arrow keys move the paddle, and the SPACE key launches the ball.
8. Update the ball position continuously using Tkinter's after() method (about 60 frames per second) to create the animation.
9. Detect collisions with the left, right and top walls and reverse the ball's direction accordingly.
10. Detect collision with the paddle and change the bounce angle according to where the ball hits the paddle.
11. Detect collision with a brick, remove that brick, add points (higher rows give more points) and bounce the ball on the axis with the smaller overlap.
12. If the ball falls below the paddle, reduce one life and place the ball back on the paddle; when lives reach zero display the GAME OVER message with the final score.
13. If all bricks are destroyed, display the YOU WIN message with the final score.
14. Provide Restart Game and Exit buttons to restart the game or close the application.
15. Execute the game and verify paddle movement, ball bouncing, scoring, lives, game-over, win and restart functionality.
16. Re-create the same game logic in HTML5 Canvas and JavaScript as a single file, brick_breaker.html, so that it can be played directly in any web browser.

DESIGN OF THE GAME LOGIC

Component	Description
Game board	600 x 500 pixel Canvas; every frame the board is cleared and everything is redrawn.
Bricks	50 bricks (5 rows x 10 columns), each 60 x 24 pixels. Row colours: red, orange, yellow, green, blue.
Paddle	100 x 14 pixels, clamped between the left and right walls, moves 28 px per key press or follows the mouse.
Ball	Radius 8 px, speed 5 px per frame. Starts stuck to the paddle in the "ready" state.
Game states	ready (waiting for SPACE), playing, over (no lives left) and won (no bricks left).
Scoring	Top row bricks = 50 points, then 40, 30, 20, and 10 for the bottom row.
Lives	Player starts with 3 lives; one is lost each time the ball falls below the board.
Paddle bounce	Horizontal speed = (hit offset from paddle centre) x ball speed x 1.2, so hitting the edges gives sharper angles.

PROGRAM CODE (PYTHON - brick_breaker.py)

```
import tkinter as tk
import random

W, H, PW, R, SP = 600, 500, 100, 8, 5          # board, paddle width, ball radius, speed
COLORS = ["#EF4444", "#F97316", "#FACC15", "#22C55E", "#38BDF8"]
s = {}                                          # game state

root = tk.Tk()
root.title("BRICK BREAKER")
root.resizable(False, False)
root.configure(bg="#172554")
tk.Label(root, text="BRICK BREAKER", font=("Arial Black", 22),
         bg="#172554", fg="white").pack(pady=10)
info = tk.Frame(root, bg="#172554")
info.pack()
score_lbl = tk.Label(info, font=("Arial", 16, "bold"), bg="#172554", fg="#FACC15")
lives_lbl = tk.Label(info, font=("Arial", 16, "bold"), bg="#172554", fg="#F87171")
score_lbl.pack(side="left", padx=25)
lives_lbl.pack(side="left", padx=25)
cv = tk.Canvas(root, width=W, height=H, bg="#0F172A",
               highlightthickness=2, highlightbackground="#38BDF8")
cv.pack(padx=20, pady=10)
bf = tk.Frame(root, bg="#172554")
bf.pack(pady=10)
tk.Button(bf, text="RESTART GAME", font=("Arial", 12, "bold"), bg="#22C55E",
          command=lambda: restart()).pack(side="left", padx=10)
tk.Button(bf, text="EXIT", font=("Arial", 12, "bold"), bg="#EF4444", fg="white",
          command=root.destroy).pack(side="left", padx=10)


def restart():
    s.update(score=0, lives=3, px=(W - PW) // 2,
             bricks=[(c * 60, 50 + r * 24, r) for r in range(5) for c in range(10)])
    reset_ball()


def reset_ball():
    s.update(state="ready", bx=s["px"] + PW // 2, by=H - 40 - R, vx=0, vy=0)


def move(x):
    if s["state"] in ("over", "won"):
        return
    s["px"] = max(0, min(W - PW, x))
    if s["state"] == "ready":
        s["bx"] = s["px"] + PW // 2


def launch(e=None):
    if s["state"] == "ready":
        s.update(state="playing", vx=random.choice([-3, 3]), vy=-SP)


def step():
    s["bx"] += s["vx"]
    s["by"] += s["vy"]
    x, y = s["bx"], s["by"]
    if x < R or x > W - R:                                  # side walls
        s["vx"] *= -1
        s["bx"] = x = max(R, min(W - R, x))
    if y < R:                                               # top wall
        s["vy"] = abs(s["vy"])
```

```
    px = s["px"]
    if s["vy"] > 0 and H - 40 <= y + R <= H - 20 and px - R <= x <= px + PW + R:   # paddle
        s["vx"] = (x - px - PW / 2) / (PW / 2) * SP * 1.2
        s["vy"], s["by"] = -SP, H - 40 - R
    for b in s["bricks"]:                                       # bricks
        bx, by, r = b
        if bx - R <= x <= bx + 60 + R and by - R <= y <= by + 24 + R:
            s["bricks"].remove(b)
            s["score"] += (5 - r) * 10
            if min(x + R - bx, bx + 60 - x + R) < min(y + R - by, by + 24 - y + R):
                s["vx"] *= -1
            else:
                s["vy"] *= -1
            break
    if not s["bricks"]:
        s["state"] = "won"
    elif y - R > H:                                             # ball lost
        s["lives"] -= 1
        reset_ball() if s["lives"] else s.update(state="over")


def draw():
    cv.delete("all")
    score_lbl.config(text=f"SCORE: {s['score']}")
    lives_lbl.config(text=f"LIVES: {s['lives']}")
    for x, y, r in s["bricks"]:
        cv.create_rectangle(x + 1, y + 1, x + 59, y + 23, fill=COLORS[r], outline="#0F172A")
    cv.create_rectangle(s["px"], H - 40, s["px"] + PW, H - 26,
                        fill="#86EFAC", outline="#14532D")
    cv.create_oval(s["bx"] - R, s["by"] - R, s["bx"] + R, s["by"] + R,
                   fill="white", outline="")
    if s["state"] == "ready":
        cv.create_text(W // 2, H // 2 + 60, fill="#94A3B8", font=("Arial", 13),
                       text="Move: Mouse / Arrow keys     Launch: SPACE")
    elif s["state"] in ("over", "won"):
        cv.create_rectangle(130, H // 2 - 70, 470, H // 2 + 70,
                            fill="#172554", outline="#38BDF8", width=2)
        cv.create_text(W // 2, H // 2 - 25, font=("Arial Black", 27),
                       text="GAME OVER" if s["state"] == "over" else "YOU WIN!",
                       fill="#EF4444" if s["state"] == "over" else "#FACC15")
        cv.create_text(W // 2, H // 2 + 25, text=f"Final Score: {s['score']}",
                       font=("Arial", 18), fill="white")


def loop():
    if s["state"] == "playing":
        step()
    draw()
    root.after(16, loop)


root.bind("<Left>", lambda e: move(s["px"] - 28))
root.bind("<Right>", lambda e: move(s["px"] + 28))
root.bind("<space>", launch)
cv.bind("<Motion>", lambda e: move(e.x - PW // 2))
restart()
loop()
root.mainloop()
```

OUTPUT:

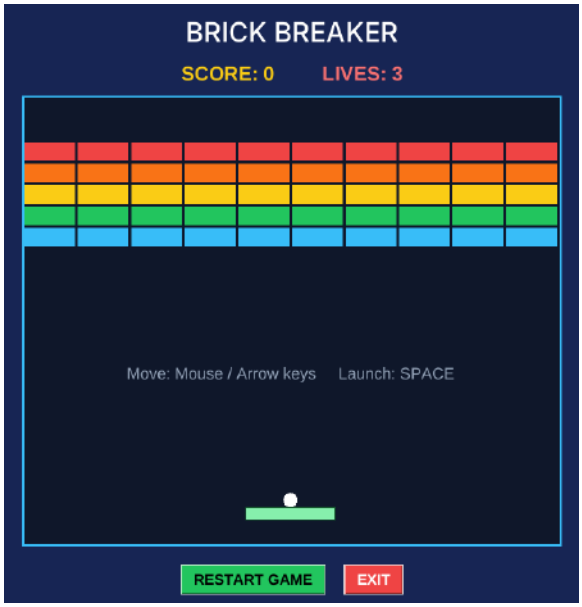

Fig 1. Game start - ball on the paddle, 3 lives

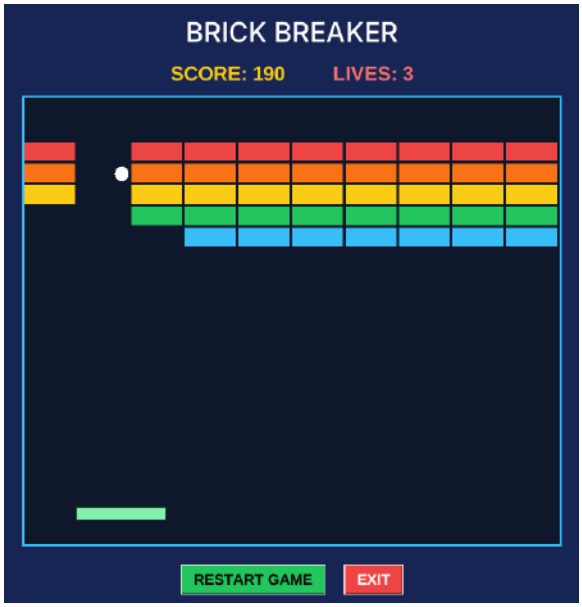

Fig 2. During play - bricks destroyed, score 190

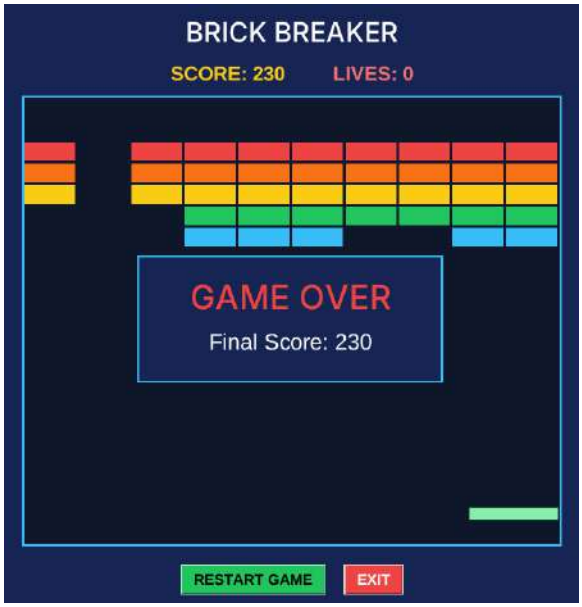

Fig 3. Game over - no lives left

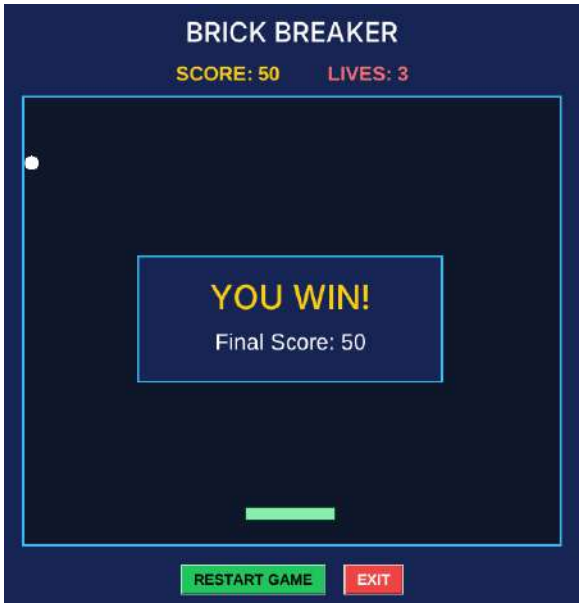

Fig 4. Win screen - all bricks destroyed

TESTING

1. The Python program was run on a virtual display with a scripted player that moves the paddle under the ball. The score increased as bricks were destroyed (score 190 after 914 frames) and lives stayed at 3 while the ball was returned.
2. The paddle was moved away from the ball: one life was lost and the ball was reset to the paddle; after three misses the GAME OVER screen appeared with the final score.
3. Restart reset the score to 0, lives to 3 and the bricks to 50.
4. With one brick left, hitting it triggered the YOU WIN state.
5. The HTML version was opened in a Chromium browser: it loaded with no console errors, the mouse and SPACE key controls worked, the score increased, lives decreased on misses, game

over appeared after the third miss, and RESTART restored the initial state.

HTML VERSION

The file brick_breaker.html contains the complete game (HTML, CSS and JavaScript) in one file. It uses the same settings, brick layout, scoring, lives and collision rules as the Python program, with a fixed 60 FPS timestep. It supports mouse, keyboard and touch controls, so it can be opened directly in any modern browser without installing Python.

RESULT:

Thus, an interactive 2D Brick Breaker game was developed using Python Tkinter with graphical game design, mouse and keyboard-based paddle control, ball physics, brick collision detection, automatic score and lives calculation, win and game-over handling, restart functionality, and exit controls. The program demonstrates event-driven programming, GUI design, animation, conditional statements, lists, and basic game development concepts. The same game was also implemented in HTML5 for playing in a web browser.